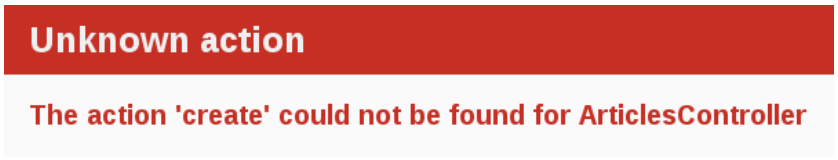
```
edit_article GET    /articles/:id/edit(.:format) articles#edit
     article GET    /articles/:id(.:format)      articles#show
             PATCH  /articles/:id(.:format)      articles#update
             PUT    /articles/:id(.:format)      articles#update
             DELETE /articles/:id(.:format)      articles#destroy
        root GET    /                            welcome#index
```

The `articles_path` helper tells Rails to point the form to the URI Pattern associated with the `articles` prefix; and the form will (by default) send a `POST` request to that route. This is associated with the `create` action of the current controller, the `ArticlesController`.

With the form and its associated route defined, you will be able to fill in the form and then click the submit button to begin the process of creating a new article, so go ahead and do that. When you submit the form, you should see a familiar error:



You now need to create the `create` action within the `ArticlesController` for this to work.

5.3 Creating articles

To make the "Unknown action" go away, you can define a `create` action within the `ArticlesController` class in `app/controllers/articles_controller.rb`, underneath the `new`action, as shown:

```
class ArticlesController < ApplicationController
  def new
  end

  def create
  end
end
```

If you re-submit the form now, you'll see another familiar error: a template is missing. That's ok, we can ignore that for now. What the `create` action should be doing is saving our new article to the database.

When a form is submitted, the fields of the form are sent to Rails as *parameters*. These parameters can then be referenced inside the controller actions, typically to perform a particular task. To see what these parameters look like, change the `create` action to this:

```
def create
  render plain: params[:article].inspect
end
```

The `render` method here is taking a very simple hash with a key of `plain` and value of `params[:article].inspect`. The `params` method is the object which represents the parameters (or fields) coming in from the form. The `params` method returns an `ActiveSupport::HashWithIndifferentAccess` object, which allows you to access the keys of the hash using either strings or symbols. In this situation, the only parameters that matter are the ones from the form.

Ensure you have a firm grasp of the `params` method, as you'll use it fairly regularly. Let's consider an example URL: **http://www.example.com/?username=dhh&email=dhh@email.com**. In this URL, `params[:username]` would equal "dhh" and `params[:email]` would equal "dhh@email.com".

If you re-submit the form one more time you'll now no longer get the missing template error. Instead, you'll see something that looks like the following: